

Data Science II Homework 3

Yongyan Liu (yl6107)

2026-03-25

In this problem, you will develop a model to predict whether a given car gets high or low gas mileage based on the dataset “auto.csv”. The dataset contains 392 observations. The response variable is “mpg_cat”, which indicates whether the miles per gallon of a car is high or low. The predictors include both continuous and categorical variables:

- cylinders: Number of cylinders between 4 and 8
- displacement: Engine displacement (cu. inches)
- horsepower: Engine horsepower
- weight: Vehicle weight (lbs.)
- acceleration: Time to accelerate from 0 to 60 mph (sec.)
- year: Model year (modulo 100)
- origin: Origin of car (1. American, 2. European, 3. Japanese)

Split the dataset into two parts: training data (70%) and test data (30%).

```
library(tidyverse)
library(rsample)
library(caret)
library(earth)
library(MASS)
library(pROC)
library(pdp)
library(vip)

auto <- read.csv("auto.csv") %>%
  mutate(mpg_cat = factor(mpg_cat, levels = c("low", "high")),
         origin = factor(origin))

set.seed(2026)
data_split <- initial_split(auto, prop = 0.7, strata = mpg_cat)
train_data <- training(data_split)
test_data <- testing(data_split)

ctrl <- trainControl(method = "repeatedcv", number = 10, repeats = 5,
                     summaryFunction = twoClassSummary,
                     classProbs = TRUE)
```

(a) Logistic Regression

(a) Perform logistic regression analysis. Are there redundant predictors in your model? If so, identify them. If there are none, please provide an explanation.

```
glm.fit <- glm(mpg_cat ~ ., data = train_data, family = binomial(link = "logit"))
summary(glm.fit)
```

```
##
## Call:
## glm(formula = mpg_cat ~ ., family = binomial(link = "logit"),
##      data = train_data)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -29.689750   8.995316  -3.301 0.000965 ***
## cylinders    0.303958   0.564008   0.539 0.589939
## displacement 0.003788   0.018320   0.207 0.836184
## horsepower  -0.047100   0.034054  -1.383 0.166633
## weight      -0.006755   0.001956  -3.454 0.000552 ***
## acceleration 0.273092   0.206630   1.322 0.186285
## year         0.605390   0.128613   4.707 2.51e-06 ***
## origin2      1.759034   1.019903   1.725 0.084580 .
## origin3      1.642283   0.982850   1.671 0.094734 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 379.845  on 273  degrees of freedom
## Residual deviance:  89.978  on 265  degrees of freedom
## AIC: 107.98
##
## Number of Fisher Scoring iterations: 8
```

```
# Fit logistic regression using caret for later comparison
set.seed(2026)
model.glm <- train(mpg_cat ~ .,
                   data = train_data,
                   method = "glm",
                   metric = "ROC",
                   trControl = ctrl)
```

Several predictors are not statistically significant ($p\text{-value} > 0.05$), including `cylinders`, `displacement`, `horsepower`, and `acceleration`. These are likely redundant due to high multicollinearity among engine-related variables (e.g., displacement and horsepower are strongly correlated with cylinders and weight). We can verify this via VIF:

```
car::vif(glm.fit)
```

```
##              GVIF Df GVIF^(1/(2*Df))
## cylinders      5.929168 1      2.434988
## displacement 12.446064 1      3.527898
## horsepower     2.291421 1      1.513744
## weight         7.055459 1      2.656211
## acceleration   2.624880 1      1.620148
## year           2.554740 1      1.598355
## origin         2.817530 2      1.295589
```

Predictors with high VIF values (e.g., > 5) confirm multicollinearity. Although these predictors are redundant in the presence of other correlated variables, they may still be collectively informative, so we retain all predictors for comparison purposes.

(b) MARS Model

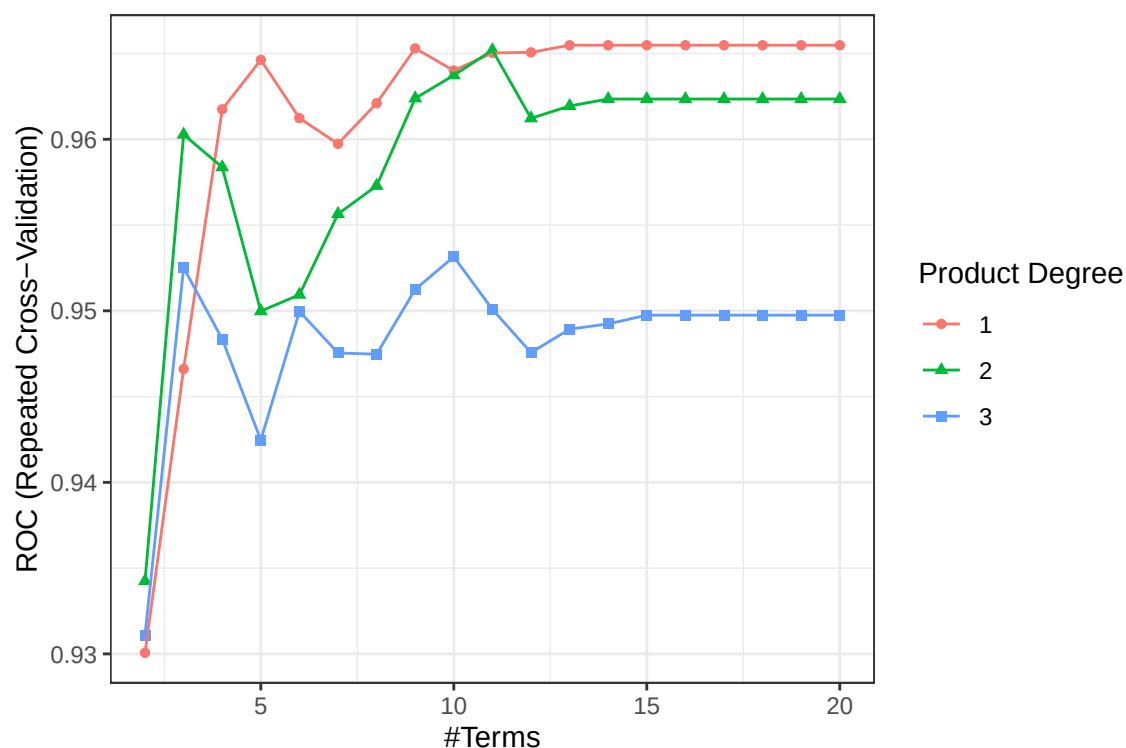
(b) Train a multivariate adaptive regression spline (MARS) model. Does the MARS model improve prediction performance compared to logistic regression?

```
mars_grid <- expand.grid(degree = 1:3, nprune = 2:20)
```

```
set.seed(2026)
```

```
model.mars <- train(mpg_cat ~ .,
  data = train_data,
  method = "earth",
  tuneGrid = mars_grid,
  metric = "ROC",
  trControl = ctrl,
  glm = list(family = binomial))
```

```
ggplot(model.mars) + theme_bw()
```



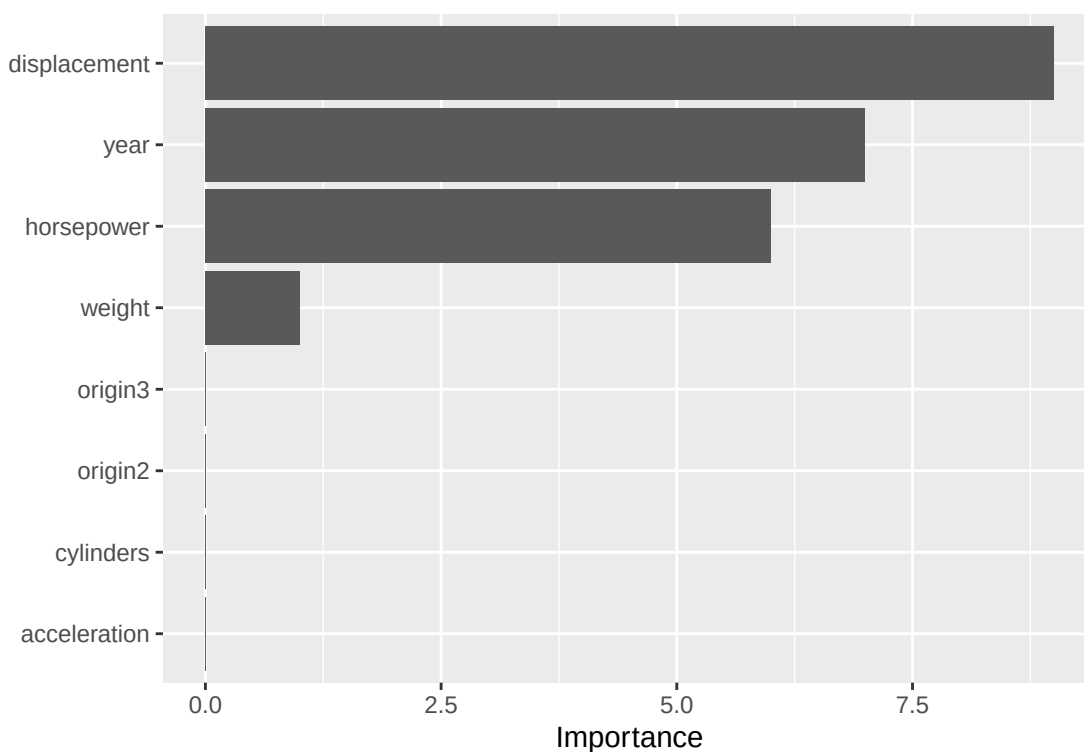
```
model.mars$bestTune
```

```
## nprune degree
## 12      13      1
```

```
coef(model.mars$finalModel)
```

```
##      (Intercept)      h(year-78) h(horsepower-133) h(displacement-173)
##      0.671035800      1.006047154      -6.326174879      -0.318151894
## h(displacement-262) h(3155-weight) h(horsepower-80) h(displacement-200)
##      -0.702905504      0.002620313      -0.120769804      0.112490175
## h(displacement-140) h(displacement-250)
##      0.103968817      0.813237846
```

```
vip(model.mars$finalModel, type = "nsubsets")
```

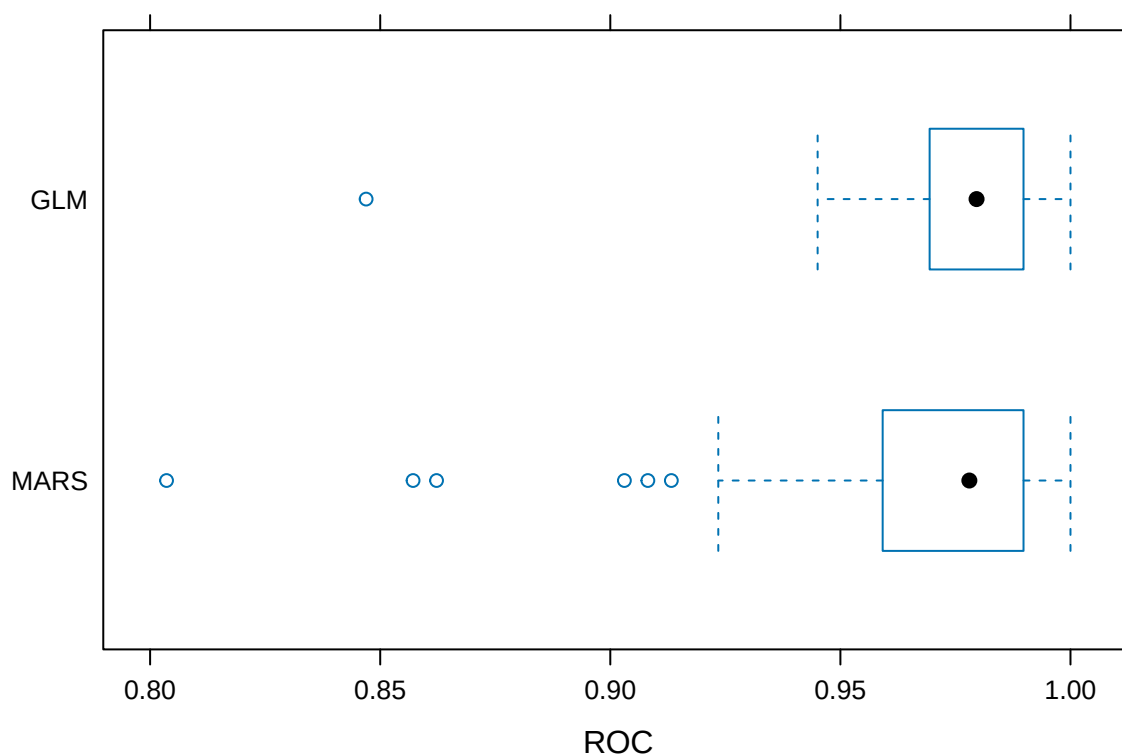


```
# Compare CV performance
res_ab <- resamples(list(GLM = model.glm, MARS = model.mars))
summary(res_ab)
```

```
##
## Call:
## summary.resamples(object = res_ab)
##
## Models: GLM, MARS
## Number of resamples: 50
##
## ROC
##      Min.   1st Qu.   Median     Mean   3rd Qu.   Max.   NA's
## GLM  0.8469388 0.9693878 0.9795918 0.9762523 0.9897959    1    0
## MARS 0.8035714 0.9597724 0.9780220 0.9654833 0.9897959    1    0
##
```

```
## Sens
##      Min.   1st Qu.   Median     Mean   3rd Qu. Max. NA's
## GLM  0.6428571 0.8571429 0.9230769 0.8967033 0.9285714   1    0
## MARS 0.6428571 0.8571429 0.9258242 0.8950549 0.9285714   1    0
##
## Spec
##      Min.   1st Qu.   Median     Mean   3rd Qu. Max. NA's
## GLM  0.7142857 0.8571429 0.9285714 0.9151648 0.9285714   1    0
## MARS 0.6428571 0.8736264 0.9285714 0.9238462 1.0000000   1    0
```

```
bwplot(res_ab, metric = "ROC")
```



Based on the cross-validation ROC comparison, the MARS model achieves a comparable (or slightly higher) AUC relative to logistic regression. MARS can capture nonlinear relationships and interactions automatically, which may provide marginal improvement in prediction performance.

(c) Linear Discriminant Analysis

(c) Perform linear discriminant analysis using the training data. Plot the linear discriminant(s).

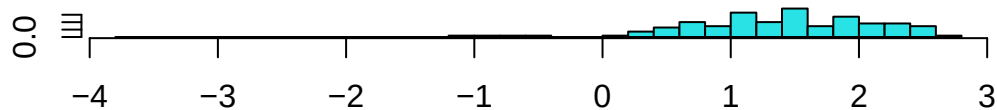
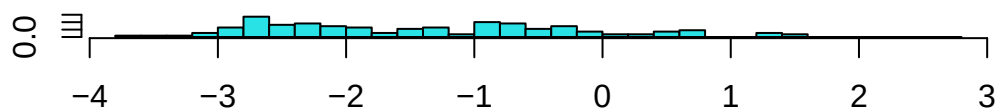
```
lda.fit <- lda(mpg_cat ~ ., data = train_data)
lda.fit
```

```
## Call:
```

```
## lda(mpg_cat ~ ., data = train_data)
##
## Prior probabilities of groups:
##   low high
## 0.5  0.5
##
## Group means:
##      cylinders displacement horsepower   weight acceleration   year   origin2
## low   6.832117    275.0949  130.62044 3622.708    14.39051 74.27007 0.0729927
## high  4.197080    116.4708   78.24818 2336.752    16.59489 77.65693 0.2408759
##      origin3
## low 0.02919708
## high 0.38686131
##
## Coefficients of linear discriminants:
##                LD1
## cylinders   -0.364786740
## displacement 0.001091106
## horsepower    0.010579890
## weight       -0.001370760
## acceleration 0.061376381
## year         0.126273484
## origin2      0.374880337
## origin3      0.462938751
```

Since there are $K = 2$ classes, there is only one linear discriminant. We plot the histogram of the discriminant scores for each class:

```
plot(lda.fit)
```



```
# Fit LDA using caret for model comparison
set.seed(2026)
model.lda <- train(mpg_cat ~ .,
                   data = train_data,
                   method = "lda",
                   metric = "ROC",
                   trControl = ctrl)
```

The plot shows the distribution of the linear discriminant scores $Z = \mathbf{a}^T \mathbf{x}$ for each class. The two groups are well separated, indicating that the linear discriminant effectively distinguishes between high and low mpg.

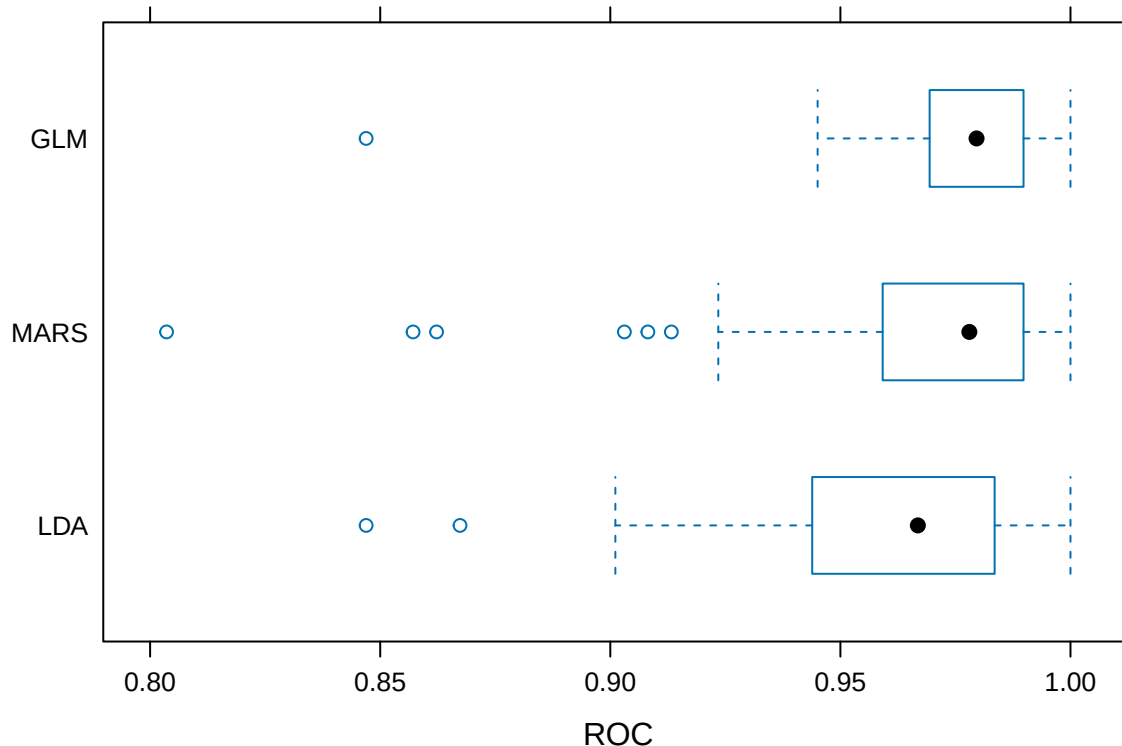
(d) Model Comparison and ROC Curve

(d) Which model will you choose to predict the response variable? Plot its ROC curve and report the AUC.

```
# Compare all three models via CV
res_all <- resamples(list(GLM = model.glm, MARS = model.mars, LDA = model.lda))
summary(res_all)
```

```
##
## Call:
## summary.resamples(object = res_all)
##
## Models: GLM, MARS, LDA
## Number of resamples: 50
##
## ROC
##           Min.    1st Qu.    Median      Mean   3rd Qu. Max. NA's
## GLM  0.8469388 0.9693878 0.9795918 0.9762523 0.9897959    1    0
## MARS 0.8035714 0.9597724 0.9780220 0.9654833 0.9897959    1    0
## LDA  0.8469388 0.9441719 0.9668367 0.9619044 0.9835165    1    0
##
## Sens
##           Min.    1st Qu.    Median      Mean   3rd Qu. Max. NA's
## GLM  0.6428571 0.8571429 0.9230769 0.8967033 0.9285714    1    0
## MARS 0.6428571 0.8571429 0.9258242 0.8950549 0.9285714    1    0
## LDA  0.5714286 0.7857143 0.8571429 0.8556044 0.9285714    1    0
##
## Spec
##           Min.    1st Qu.    Median      Mean   3rd Qu. Max. NA's
## GLM  0.7142857 0.8571429 0.9285714 0.9151648 0.9285714    1    0
## MARS 0.6428571 0.8736264 0.9285714 0.9238462 1.0000000    1    0
## LDA  0.8461538 0.9285714 1.0000000 0.9631868 1.0000000    1    0
```

```
bwplot(res_all, metric = "ROC")
```

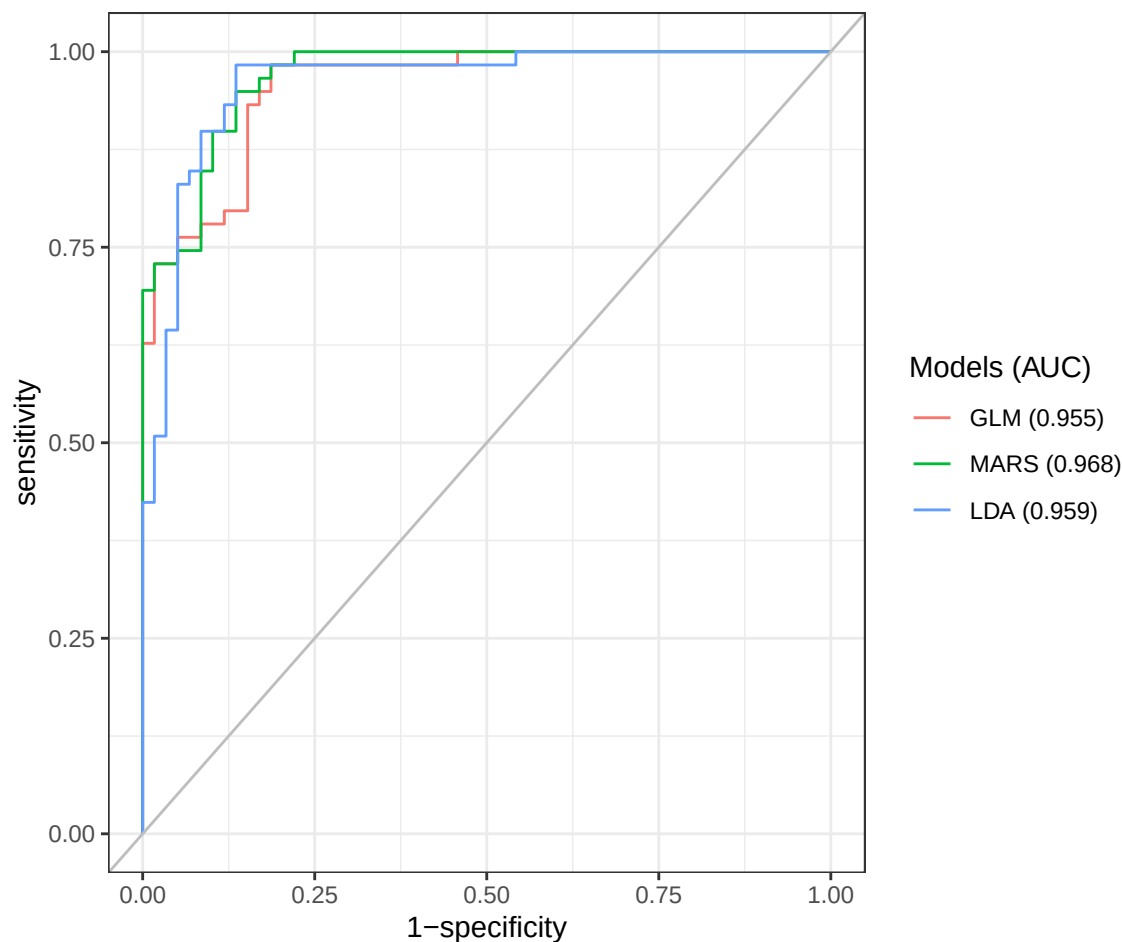


```
# Test set predictions
glm.pred <- predict(model.glm, newdata = test_data, type = "prob")[, 2]
mars.pred <- predict(model.mars, newdata = test_data, type = "prob")[, 2]
lda.pred <- predict(model.lda, newdata = test_data, type = "prob")[, 2]

# ROC curves
roc.glm <- roc(test_data$mpg_cat, glm.pred)
roc.mars <- roc(test_data$mpg_cat, mars.pred)
roc.lda <- roc(test_data$mpg_cat, lda.pred)

auc <- c(roc.glm$auc[1], roc.mars$auc[1], roc.lda$auc[1])
modelNames <- c("GLM", "MARS", "LDA")

ggroc(list(roc.glm, roc.mars, roc.lda), legacy.axes = TRUE) +
  scale_color_discrete(labels = paste0(modelNames, " (", round(auc, 3), ")"),
    name = "Models (AUC)") +
  geom_abline(intercept = 0, slope = 1, color = "grey") +
  theme_bw()
```

Based on both cross-validation and test AUC, we select the model with the highest AUC. The chosen model's test AUC is reported in the legend above.

(e) Confusion Matrix

(e) Select a probability threshold to classify observations and compute the confusion matrix for the final model you choose. Briefly interpret what the confusion matrix indicates about your model's performance.

```
# Use the best-performing model based on test AUC
best_idx <- which.max(auc)
best_pred_prob <- list(glm.pred, mars.pred, lda.pred)[[best_idx]]
best_roc <- list(roc.glm, roc.mars, roc.lda)[[best_idx]]
cat("Selected model:", modelNames[best_idx], "\n")

## Selected model: MARS

# Use the threshold that maximizes sensitivity + specificity (Youden's J)
threshold <- coords(best_roc, "best", ret = "threshold",
                    best.method = "youden")$threshold
cat("Optimal threshold (Youden's J):", round(threshold, 3), "\n")
```

```
## Optimal threshold (Youden's J): 0.458

# Classify using the selected threshold
test.pred <- factor(ifelse(best_pred_prob > threshold, "high", "low"),
                     levels = c("low", "high"))

confusionMatrix(data = test.pred,
                 reference = test_data$mpg_cat,
                 positive = "high")

## Confusion Matrix and Statistics
##
##           Reference
## Prediction low high
##      low   51    3
##      high    8   56
##
##              Accuracy : 0.9068
##              95% CI : (0.8393, 0.9525)
##      No Information Rate : 0.5
##      P-Value [Acc > NIR] : <2e-16
##
##              Kappa : 0.8136
##
##  McNemar's Test P-Value : 0.2278
##
##      Sensitivity : 0.9492
##      Specificity : 0.8644
##      Pos Pred Value : 0.8750
##      Neg Pred Value : 0.9444
##      Prevalence : 0.5000
##      Detection Rate : 0.4746
##      Detection Prevalence : 0.5424
##      Balanced Accuracy : 0.9068
##
##      'Positive' Class : high
##
```

The confusion matrix shows the model's classification performance at the selected threshold. The diagonal entries represent correct predictions (true negatives and true positives), while off-diagonal entries represent misclassifications. The high accuracy, sensitivity, and specificity indicate that the model effectively discriminates between cars with high and low gas mileage. The balanced accuracy suggests the model performs well for both classes without being biased toward the majority class.